BLAMELESS RETRO · 9 JUNE 2026

The 2 June payments outage

What happened, what we learned, and the four changes we are making.

On 2 June, payments were down for 47 minutes — here is what we learned.

A configuration change took the payments service offline during the evening peak. We recovered in 47 minutes. This retro is blameless: the goal is to understand the system that let it happen and fix it, not to find a person to blame.

The blameless frame: we fix systems, not people.

Good people made reasonable decisions with the information they had. The outage happened because our system let a single unreviewed change reach production at the worst possible time. We focus on the guardrails that were missing, not the hands on the keyboard.

- Every action item targets a process or a safeguard, never an individual.
- The engineer who made the change helped write this retro.

2 JUNE 2026 · ALL TIMES LOCAL

How the incident unfolded, minute by minute.

18:42

Config change deployed

A routing rule was pushed directly to production, bypassing staging.

18:44

Payments begin failing

at-risk

Error rate spikes to 100%; checkout returns errors.

18:51

Incident declared

live

On-call paged after customer reports; war room opened.

19:29

Service restored

done

The change was rolled back and traffic recovered fully.

CUSTOMER IMPACT

47 min

of failed checkouts during evening peak — roughly 12,000 transactions affected, all later recoverable.

What went well, and what went poorly.

Went well · Detection.

Customer reports reached us within two minutes

Went well · Response.

War room and rollback ran cleanly once declared

Went poorly · Prevention.

The change skipped staging entirely

Went poorly · Alerting.

We learned from customers before our own monitors

Root cause: a manual change path that bypassed every safeguard.

The routing rule was changed through a console that writes straight to production. There was no required review, no staging step, and no alert on the resulting error spike — so three independent safeguards were all absent on the same path.

- The console predates our deployment pipeline and was never retired.
- Error-rate alerting covered the API but not the routing layer.

RESPONSE · THIS INCIDENT VS OUR TARGETS

How our response compared to our incident targets.

Targets from the incident-response runbook, last revised Q1 2026.

---

9 min
TO DECLARE (TARGET 5)

38 min
TO MITIGATE (TARGET 30)

47 min
TOTAL DOWNTIME

100%
TRANSACTIONS RECOVERED

The four changes we are making, with owners.

STEP 01

Retire the console

Direct production writes are disabled this week; all changes route through the pipeline. Owner: SRE.

STEP 02

Require staging

No config change reaches production without a staging deploy and a second reviewer. Owner: Platform.

STEP 03

Alert the routing layer

Error-rate alerts now cover routing, paging on-call within 60 seconds. Owner: SRE.

STEP 04

Rehearse rollback

A monthly game-day drills the rollback path so it stays fast. Owner: Eng leads.

Action-item status as of this retro.

✓ Incident timeline reconstructed	complete
✓ Customer comms sent	complete
– Direct-write console disabled	in progress
○ Staging gate enforced in pipeline	this sprint
○ Routing-layer alerts live	this sprint
○ First rollback game-day scheduled	July

INCIDENT REVIEW · SRE@EXAMPLE.COM

We fix the system so this can't happen the same way twice.